

Reflection Log

Noorinder

Credit 3

Assignment Name: Account, PersonalAcct, BusinessAcct, Bank

How has your program changed from planning to coding to now?

Initially, the program was planned to simulate a simple banking system with two account types: `PersonalAcct` and `BusinessAcct`. The base class `Account` would encapsulate shared functionality like depositing money and enforcing balance checks. Subclasses would add unique features, such as penalties for low balances. The `Bank` class served as the user interface for performing actions like deposits, withdrawals, and viewing account details.

While coding a few challenges arose that required adjustments to improve functionality

1. Missing Validation for Withdrawals

- **Problem:** Initially, the `withdraw()` method in both subclasses didn't validate whether the withdrawal amount was positive. This allowed users to attempt invalid withdrawals (e.g., entering `-50` to increase their balance).
- **Fix:** I added a check to ensure withdrawal amounts were positive.

Reflection: This fix improved data integrity and prevented users from entering unrealistic amounts.

Code Example:

```
if (amount <= 0) {  
    throw new IllegalArgumentException("Withdrawal amount must be positive.");  
}
```

2. Incorrect Penalty Application

- **Problem:** The penalty for falling below the minimum balance was applied even if the withdrawal couldn't be completed due to insufficient funds. This caused the balance to decrease unnecessarily.

- **Fix:** I adjusted the `withdraw()` logic to apply penalties only if the withdrawal was successful.

Reflection: This ensured penalties were applied correctly, improving the program's fairness and accuracy.

Code Example:

```
if (balance - amount >= 0) {  
    if (balance - amount < 100) { // Apply penalty for PersonalAcct  
        balance -= 2;  
    }  
    balance -= amount;  
} else {  
    throw new IllegalArgumentException("Insufficient funds for withdrawal.");  
}
```

3. Handling Invalid Account Types

- **Problem:** Entering an invalid account type caused the program to display incorrect behavior or crash. This was due to a lack of input validation in the `switch` statement.
- **Fix:** I added a `default` case to handle invalid inputs and re-prompt the user.

Reflection: This change made the program more user-friendly and prevented crashes caused by unexpected input.

Code Example:

```
switch (acctType) {  
    case 1:  
        account = personalAcct;  
        break;  
    case 2:  
        account = businessAcct;  
        break;  
    default:  
        System.out.println("Invalid Account type! Please try again.");  
        continue; // Re-prompt the user  
}
```

4. Preventing Negative Initial Balances

- **Problem:** The program allowed negative balances during account creation if the user bypassed validation logic. This caused inconsistencies in account behavior.

- **Fix:** I added a validation check in the `Account` constructor to ensure the initial balance is non-negative.

Reflection: This improved the reliability of the program by enforcing realistic initial conditions.

Code Example:

```
public Account(double balance) {  
    if (balance < 0) {  
        throw new IllegalArgumentException("Initial balance cannot be negative.");  
    }  
    this.balance = balance;  
}
```

5. No Confirmation for Quit

- **Problem:** The program exited immediately when the user selected the quit option without confirming their choice. This could lead to accidental termination of the session.
- **Fix:** I added a confirmation prompt to ensure users intended to quit.

Reflection: This improved the program's interactivity and reduced the likelihood of accidental exits.

Code Example:

```
if (action.equalsIgnoreCase("Q")) {  
    System.out.print("Are you sure you want to quit? (yes/no): ");  
    String confirmation = input.next();  
    if (!confirmation.equalsIgnoreCase("yes")) {  
        continue; // Do not exit if the user changes their mind  
    }  
}
```